

1. Write a C program to store n elements in an array using dynamic memory allocation and find the largest number.

Program

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, i, *a, largest;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    a = (int *)malloc(n * sizeof(int));

    if (a == NULL)
    {
        printf("Memory allocation failed");
        return 1;
    }

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    largest = a[0];

    for (i = 1; i < n; i++)
    {
        if (a[i] > largest)
            largest = a[i];
    }

    printf("Largest number = %d\n", largest);

    free(a);
    return 0;
}
```

Output

```
Enter number of elements: 5
Enter 5 elements:
12 45 7 89 23
Largest number = 89
```

2. Print an array in reverse order using pointers.

Program

```
#include <stdio.h>

int main()
{
    int a[100], n, i;
    int *p;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    p = a + n - 1;

    printf("Array in reverse order:\n");
    for (i = n - 1; i >= 0; i--)
        printf("%d ", *p--);

    printf("\n");
    return 0;
}
```

Output

```
Enter number of elements: 5
Enter 5 elements:
10 20 30 40 50
Array in reverse order:
50 40 30 20 10
```

3. Write a C program to store information of students using structures. Sort them by Name.

Program

```
#include <stdio.h>
#include <string.h>

struct Student
{
    int roll;
    char name[50];
    float mark;
};

int main()
{
    struct Student s[50], temp;
    int n, i, j;

    printf("Enter number of students: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("\nStudent %d\n", i + 1);
        printf("Roll number: ");
        scanf("%d", &s[i].roll);
        printf("Name: ");
        scanf("%s", s[i].name);
        printf("Mark: ");
        scanf("%f", &s[i].mark);
    }

    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (strcmp(s[i].name, s[j].name) > 0)
            {
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }

    printf("\nStudents sorted by Name:\n");
    for (i = 0; i < n; i++)
        printf("%d %s %.2f\n", s[i].roll, s[i].name, s[i].mark);

    return 0;
}
```

Output

Enter number of students: 3

Student 1
Roll number: 101
Name: Ravi
Mark: 82

Student 2
Roll number: 102
Name: Arun
Mark: 91
Student 3

Roll number: 103

Name: Bala

Mark: 76

Students sorted by Name:

102 Arun 91.00

103 Bala 76.00

101 Ravi 82.00

4. Write a C program to add two distances given in inch-feet system using structures.

Program

```
#include <stdio.h>

struct Distance
{
    int feet;
    int inch;
};

int main()
{
    struct Distance d1, d2, sum;

    printf("Enter first distance (feet inch): ");
    scanf("%d %d", &d1.feet, &d1.inch);

    printf("Enter second distance (feet inch): ");
    scanf("%d %d", &d2.feet, &d2.inch);

    sum.feet = d1.feet + d2.feet;
    sum.inch = d1.inch + d2.inch;

    if (sum.inch >= 12)
    {
        sum.feet += sum.inch / 12;
        sum.inch %= 12;
    }

    printf("Sum = %d feet %d inch\n", sum.feet, sum.inch);

    return 0;
}
```

Output

```
Enter first distance (feet inch): 5 8
Enter second distance (feet inch): 3 7
Sum = 9 feet 3 inch
```

5. Write a C program to add two complex numbers by passing structure to a function.

Program

```
#include <stdio.h>

struct Complex
{
    float real;
    float imag;
};

struct Complex add(struct Complex c1, struct Complex c2)
{
    struct Complex result;

    result.real = c1.real + c2.real;
    result.imag = c1.imag + c2.imag;

    return result;
}

int main()
{
    struct Complex c1, c2, sum;

    printf("Enter first complex number (real imaginary): ");
    scanf("%f %f", &c1.real, &c1.imag);

    printf("Enter second complex number (real imaginary): ");
    scanf("%f %f", &c2.real, &c2.imag);

    sum = add(c1, c2);

    printf("Sum = %.2f + %.2fi\n", sum.real, sum.imag);

    return 0;
}
```

Output

```
Enter first complex number (real imaginary): 4 5
Enter second complex number (real imaginary): 2 3
Sum = 6.00 + 8.00i
```

6. Write a C program to create an inventory management system using structures.

Program

```
#include <stdio.h>

struct Item
{
    int id;
    char name[50];
    int quantity;
    float price;
};

int main()
{
    struct Item item[50];
    int n, i;

    printf("Enter number of items: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("\nItem %d\n", i + 1);
        printf("ID: ");
        scanf("%d", &item[i].id);
        printf("Name: ");
        scanf(" %[^
]", item[i].name);
        printf("Quantity: ");
        scanf("%d", &item[i].quantity);
        printf("Price: ");
        scanf("%f", &item[i].price);
    }

    printf("\nInventory:\n");
    printf("ID\tName\t\tQuantity\tPrice\n");

    for (i = 0; i < n; i++)
    {
        printf("%d\t%-15s %d\t\t%.2f\n",
            item[i].id, item[i].name,
            item[i].quantity, item[i].price);
    }

    return 0;
}
```

Output

Enter number of items: 2

Item 1

ID: 101

Name: Pen

Quantity: 20

Price: 15

Item 2

ID: 102

Name: Notebook

Quantity: 10

Price: 50

Inventory:

ID■Name■Quantity■Price

101■Pen 20■15.00

102■Notebook 10■50.00

7. Write a C program for decimal to binary, octal and hexadecimal conversion.

Program

```
#include <stdio.h>

void convert(int n, int base)
{
    char digits[] = "0123456789ABCDEF";
    char result[50];
    int i = 0;

    if (n == 0)
    {
        printf("0");
        return;
    }

    while (n > 0)
    {
        result[i++] = digits[n % base];
        n /= base;
    }

    while (i > 0)
        printf("%c", result[--i]);
}

int main()
{
    int n;

    printf("Enter a decimal number: ");
    scanf("%d", &n);

    printf("Binary = ");
    convert(n, 2);

    printf("\nOctal = ");
    convert(n, 8);

    printf("\nHexadecimal = ");
    convert(n, 16);

    printf("\n");
    return 0;
}
```

Output

```
Enter a decimal number: 25
Binary = 11001
Octal = 31
Hexadecimal = 19
```

8. Write a C program to calculate difference between two time periods.

Program

```
#include <stdio.h>

struct Time
{
    int hour;
    int minute;
    int second;
};

int main()
{
    struct Time t1, t2, diff;
    int sec1, sec2, difference;

    printf("Enter first time (hh mm ss): ");
    scanf("%d %d %d", &t1.hour, &t1.minute, &t1.second);

    printf("Enter second time (hh mm ss): ");
    scanf("%d %d %d", &t2.hour, &t2.minute, &t2.second);

    sec1 = t1.hour * 3600 + t1.minute * 60 + t1.second;
    sec2 = t2.hour * 3600 + t2.minute * 60 + t2.second;

    difference = sec1 - sec2;

    if (difference < 0)
        difference = -difference;

    diff.hour = difference / 3600;
    difference %= 3600;
    diff.minute = difference / 60;
    diff.second = difference % 60;

    printf("Difference = %02d:%02d:%02d\n",
        diff.hour, diff.minute, diff.second);

    return 0;
}
```

Output

```
Enter first time (hh mm ss): 10 30 20
Enter second time (hh mm ss): 8 45 50
Difference = 01:44:30
```

9. Write a C program to create an employee management system using structures.

Program

```
#include <stdio.h>

struct Employee
{
    int id;
    char name[50];
    float salary;
};

int main()
{
    struct Employee e[50];
    int n, i;

    printf("Enter number of employees: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("\nEmployee %d\n", i + 1);
        printf("ID: ");
        scanf("%d", &e[i].id);
        printf("Name: ");
        scanf(" %[^
]", e[i].name);
        printf("Salary: ");
        scanf("%f", &e[i].salary);
    }

    printf("\nEmployee Details:\n");
    printf("ID\tName\t\tSalary\n");

    for (i = 0; i < n; i++)
    {
        printf("%d\t%-15s %.2f\n",
            e[i].id, e[i].name, e[i].salary);
    }

    return 0;
}
```

Output

Enter number of employees: 2

Employee 1
ID: 101
Name: Arun
Salary: 25000

Employee 2
ID: 102
Name: Ravi
Salary: 30000

Employee Details:
ID■Name■Salary
101■Arun 25000.00
102■Ravi 30000.00

10. Write a C program to calculate area and perimeter of a rectangle and triangle using function pointers.

Program

```
#include <stdio.h>
#include <math.h>

float rectangle_area(float l, float b)
{
    return l * b;
}

float rectangle_perimeter(float l, float b)
{
    return 2 * (l + b);
}

float triangle_area(float a, float b, float c)
{
    float s = (a + b + c) / 2;
    return sqrt(s * (s - a) * (s - b) * (s - c));
}

float triangle_perimeter(float a, float b, float c)
{
    return a + b + c;
}

int main()
{
    float l, b, a, c;
    float (*area)(float, float);
    float (*perimeter)(float, float);

    printf("Enter rectangle length and breadth: ");
    scanf("%f %f", &l, &b);

    area = rectangle_area;
    perimeter = rectangle_perimeter;

    printf("Rectangle area = %.2f\n", area(l, b));
    printf("Rectangle perimeter = %.2f\n", perimeter(l, b));

    printf("\nEnter triangle sides: ");
    scanf("%f %f %f", &a, &b, &c);

    printf("Triangle area = %.2f\n", triangle_area(a, b, c));
    printf("Triangle perimeter = %.2f\n",
           triangle_perimeter(a, b, c));

    return 0;
}
```

Output

```
Enter rectangle length and breadth: 10 5
Rectangle area = 50.00
Rectangle perimeter = 30.00

Enter triangle sides: 3 4 5
Triangle area = 6.00
Triangle perimeter = 12.00
```